

Task Manager API

Este projeto consiste numa API REST para gestão de tarefas, desenvolvida em Node.js com Express e MySQL. A aplicação permite autenticação de utilizadores através do fluxo OAuth2 Resource Owner Password Credentials, emitindo tokens de acesso no formato JWT (JSON Web Token), e operações CRUD sobre tarefas, garantindo que cada utilizador apenas pode aceder às suas próprias informações. Para o desenvolvimento da API foi utilizada uma abordagem Code First, onde a implementação do backend e das rotas foi realizada inicialmente em código. Posteriormente, a documentação Swagger/OpenAPI foi gerada automaticamente através de anotações `@openapi` inseridas nos controllers da aplicação.

Tecnologias Utilizadas

- Node.js
 - Express.js
 - MySQL 5.7
 - Docker / Docker Compose
 - OAuth2 (Resource Owner Password Credentials Flow)
 - JWT (JSON Web Token) — biblioteca `jsonwebtoken`
 - Bcrypt — biblioteca `bcryptjs`
 - Swagger / OpenAPI 3.0 — bibliotecas `swagger-jsdoc` e `swagger-ui-express`
 - mysql2
 - dotenv
 - cors
-

Estrutura do Projeto

```
task-manager-api/  
├── api/  
│   └── openapi.yaml  
├── db/  
│   └── init.sql  
├── src/  
│   ├── config/  
│   │   └── db.js  
│   ├── controllers/  
│   │   ├── authController.js  
│   │   ├── taskController.js  
│   │   ├── categoryController.js  
│   │   └── userController.js  
│   ├── middlewares/  
│   │   └── auth.js  
│   └── routes/  
│       └── index.js
```

```
├── .dockerignore
├── .env.example
├── Dockerfile
├── docker-compose.yml
├── package.json
├── server.js
└── README.md
```

- **api/** → ficheiro OpenAPI estático para documentação e importação no Postman
- **config/** → configuração da ligação ao MySQL
- **controllers/** → lógica das rotas e queries SQL
- **middlewares/** → middleware de autenticação OAuth2/JWT
- **routes/** → definição dos endpoints da API
- **db/** → script de inicialização da base de dados

Base de Dados

A aplicação utiliza MySQL como sistema de gestão de base de dados. A base de dados é criada automaticamente através do ficheiro **init.sql**, executado pelo container Docker do MySQL durante a inicialização.

Recursos Implementados

Foram implementados 3 recursos principais:

- Utilizadores (**users**)
- Tarefas (**tasks**)
- Categorias (**categories**)

Foi também criada uma tabela intermédia **task_categories** para representar a relação entre tarefas e categorias.

Relações entre Recursos

Relação 1:N

Existe uma relação de cardinalidade **1:N** entre **users** e **tasks**.

Um utilizador pode possuir várias tarefas, enquanto cada tarefa pertence apenas a um utilizador.

Essa relação é implementada através da chave estrangeira:

```
user_id INT,
FOREIGN KEY (user_id) REFERENCES users(id)
```

Esta relação é visível no endpoint `GET /users/{id}`, que devolve o utilizador com a lista completa das suas tarefas, e no endpoint `GET /users/{id}/tasks`, que devolve as tarefas com as categorias associadas.

Autenticação e Autorização

JWT — JSON Web Token

O projeto utiliza **JWT** como formato do token de acesso. Após validação das credenciais, o servidor gera um token assinado com a biblioteca `jsonwebtoken`:

```
const access_token = jwt.sign(  
  { id: user.id, name: user.name, client_id, scope: grantedScope },  
  process.env.JWT_SECRET,  
  { expiresIn: 3600 }  
);
```

O token contém o `id`, `name`, `client_id` e `scope` do utilizador. É assinado com uma chave secreta (`JWT_SECRET`) e expira ao fim de 3600 segundos (1 hora).

OAuth2 — Resource Owner Password Credentials Flow

A autenticação segue o fluxo OAuth2 ROPC. O utilizador obtém um token através do endpoint `/oauth/token`, enviando:

- `grant_type` → `password`
- `username` → email do utilizador
- `password`
- `client_id`
- `client_secret`
- `scope` → por omissão `read write`

O servidor valida primeiro a aplicação cliente (`client_id` e `client_secret`) e depois as credenciais do utilizador, comparando a password com o hash guardado na base de dados através do `bcryptjs`. Em caso de sucesso, é devolvida uma resposta no formato OAuth2:

```
{  
  "access_token": "<token JWT>",  
  "token_type": "Bearer",  
  "expires_in": 3600,  
  "scope": "read write"  
}
```

O token é enviado pelo cliente no header HTTP:

```
Authorization: Bearer <token>
```

O middleware de autenticação (`auth.js`) verifica a existência, assinatura e expiração do token. Após validação, os dados do utilizador são injetados em `req.user`, permitindo controlar o acesso aos recursos.

Autorização por utilizador

A autorização foi implementada ao nível das queries SQL, filtrando sempre pelo `user_id` presente no token:

- um utilizador apenas consegue visualizar as suas tarefas
- um utilizador apenas consegue editar as suas tarefas
- um utilizador apenas consegue apagar as suas tarefas
- um utilizador apenas consegue aceder ao seu próprio perfil (`/users/{id}`)

Implementação OAuth2 e Comparação de Flows

A autenticação implementada segue o fluxo **Resource Owner Password Credentials (ROPC)**, no qual a aplicação cliente recolhe as credenciais do utilizador e as troca por um token de acesso. Este fluxo utiliza:

- autenticação da aplicação cliente via `client_id` e `client_secret`
- `grant_type` para identificar o tipo de fluxo
- `scopes` para indicar as permissões associadas ao token
- token de acesso JWT com tempo de expiração (`expires_in`)

No OAuth2 existem outros flows, cada um adequado a cenários diferentes:

Flow	Cenário de uso
Authorization Code	Aplicações web com servidor próprio; login com Google, GitHub, etc.
Client Credentials	Comunicação máquina a máquina, sem utilizador envolvido
Implicit	Aplicações SPA (descontinuado no OAuth 2.1)
Device Flow	Dispositivos sem browser (TV, CLI)
ROPC (este projeto)	Aplicações de confiança em que o cliente e a API são da mesma equipa

O fluxo ROPC foi escolhido por ser adequado a uma aplicação em que a mesma equipa controla tanto o cliente como a API, dispensando a complexidade de redirecionamentos e de um Authorization Server externo.

Docker

A aplicação utiliza uma arquitetura multi-container com dois containers:

- **app** — Node.js (construído a partir do `Dockerfile`)

- **db** — MySQL 5.7 (imagem oficial)

O MySQL inclui um **healthcheck** que garante que o container Node só arranca depois da base de dados estar pronta.

Execução

```
docker-compose up --build
```

Para recriar a base de dados do zero (necessário após alterações ao **init.sql**):

```
docker-compose down -v
docker-compose up --build
```

URLs disponíveis

Recurso	URL
API	http://localhost:3000
Swagger UI	http://localhost:3000/api-docs
Swagger JSON	http://localhost:3000/swagger.json

Endpoints

Método	Endpoint	Descrição	Autenticação
POST	/register	Registar novo utilizador	Pública
POST	/oauth/token	Obter token de acesso (OAuth2)	Pública
GET	/users/me	Obter perfil do utilizador autenticado	Bearer Token
GET	/users/{id}	Obter utilizador com as suas tarefas (1:N)	Bearer Token
PUT	/users/{id}	Atualizar utilizador (só o próprio)	Bearer Token
DELETE	/users/{id}	Apagar utilizador (só o próprio)	Bearer Token
GET	/users/{id}/tasks	Listar tarefas com categorias (1:N)	Bearer Token
GET	/tasks	Listar tarefas do utilizador	Bearer Token
POST	/tasks	Criar tarefa	Bearer Token
PUT	/tasks/{id}	Atualizar tarefa	Bearer Token
DELETE	/tasks/{id}	Apagar tarefa	Bearer Token

Método	Endpoint	Descrição	Autenticação
GET	/categories	Listar categorias	Bearer Token
POST	/categories	Criar categoria	Bearer Token
GET	/categories/{id}	Obter categoria por ID	Bearer Token
PUT	/categories/{id}	Atualizar categoria	Bearer Token
DELETE	/categories/{id}	Apagar categoria	Bearer Token

Segurança

A aplicação implementa:

- autenticação OAuth2 com tokens de acesso JWT
- middleware de autorização em todas as rotas privadas
- hash de passwords com bcrypt
- isolamento de recursos por `user_id` (tarefas e perfil)
- variáveis de ambiente com dotenv
- verificação de token Bearer (existência, assinatura e expiração)

O detalhe do utilizador autenticado é apresentado na consola a cada pedido recebido:

```
--- [PEDIDO RECEBIDO] ---  
Rota acessada: GET /tasks  
Utilizador Autenticado: Igor Silva (ID: 1)  
Aplicação Cliente via OAuth2: task-manager-app  
Escopos autorizados no Token: read write  
-----
```

Documentação da API

A documentação foi implementada com Swagger / OpenAPI 3.0 de duas formas complementares:

- **Swagger UI** — gerado automaticamente a partir das anotações `@openapi` nos controllers, disponível em `/api-docs`
- **openapi.yaml** — ficheiro estático em `api/openapi.yaml`, para entrega e importação no Postman

Collection Postman

O projeto inclui uma collection do Postman para facilitar os testes da API.

A collection permite testar:

- obtenção do token de acesso (OAuth2)
- autenticação com token Bearer
- CRUD de tarefas
- consulta de categorias

Para importar no Postman

1. Abre o Postman
 2. Clica em **Import**
 3. Selecciona o ficheiro `api/openapi.yaml` ou introduz o URL `http://localhost:3000/swagger.json`
 4. O Postman gera automaticamente a collection com todos os endpoints
-

Utilização de Ferramentas de IA

Durante o desenvolvimento deste projeto foram utilizadas ferramentas de Inteligência Artificial como apoio em partes específicas, nomeadamente:

- na abordagem Code First da API, como auxílio na estruturação inicial do código e das rotas
- na criação da base de dados, designadamente na geração do script `init.sql` e dos dados de teste

As restantes componentes do projeto foram desenvolvidas e validadas por mim, recorrendo à IA apenas como apoio complementar sempre que necessário.

Conclusão

Este projeto permitiu desenvolver uma API REST completa utilizando Node.js, Express e MySQL, aplicando conceitos de autenticação OAuth2, autorização por utilizador, JWT e documentação de APIs com OpenAPI 3.0. Foram implementadas operações CRUD protegidas por tokens Bearer, garantindo que cada utilizador apenas consegue aceder e modificar os seus próprios recursos.